ACADEMIC PROJECT PROGRESS & ARCHITECTURAL REPORT

StayEase — 9-Week Lifecycle & Official UML Architecture

Complete Documentation: Team Planning, SRS, Production UML Diagrams (Use Case, Class, Sequence, State Machine & ER), 4-Module Division, and Integration Testing

METHODOLOGY	ARCHITECTURE	TEST PASS RATE	CORE TECHNOLOGY
Agile Iterative (9 Academic Weeks)	Multi-Tier Spring Boot 3 MVC + Micro-Services	295+ Automated Tests (100% Pass)	Java 21 • Spring Data JPA • PostgreSQL

1. Academic Lifecycle Timeline at a Glance

Week	Academic Stage	Key Engineering Deliverables	Status in StayEase
Week 1	Team Formation + Guide Selection + Abstract	Problem Identification, Guide Approval, Project Scope & Abstract	Completed Problem & Scope Defined
Week 2	Software Requirements Specification (SRS)	Functional & Non-Functional Requirements, User Personas, Constraints	Completed Full SRS Documented
Week 3	UML System Design	Use Case, Class, Concurrency Sequence, State Machine & Deployment	Completed Visual Models Engineered
Week 4	Database Design & UI Mock-ups	3NF Relational Schema, ER Diagram, Wireframes & Thymeleaf Mockups	Completed MySQL/Postgres Schema + UI
Weeks 5–8	Module Coding (4 Individual Modules)	Student 1: Maps & Geocoding • Student 2: Booking Engine Student 3: Property Operations • Student 4: Security & Auth	Completed All 4 Modules Built & Documented
Week 9	System Integration & Testing	End-to-End Integration, 295+ Automated Tests, Docker & Cloud Deploy	Completed 100% Tests Pass + Cloud Live

🗓️ WEEK 1: Team Formation, Guide Selection & Abstract

WEEK 1

Goal: Form 4-member team, obtain faculty guide approval, and formulate the project abstract.

Problem Statement: Students and young professionals relocating to new cities suffer from 15%–30% broker commissions and manual hostel registers causing frequent double-bookings. StayEase was conceptualized to provide a zero-brokerage reservation portal with free worldwide geospatial intelligence for guests, paired with an administrative property management dashboard for owners.

📋 WEEK 2: Software Requirements Specification (SRS)

WEEK 2

Goal: Finalize 6 Functional Requirements (FRs) and 4 Non-Functional Requirements (NFRs).

- **Functional:** FR-1 (Auth & RBAC), FR-2 (Free OpenStreetMap Geocoding & 100km Radar), FR-3 (Zero-Overbooking Transactional Booking), FR-4 (Digital Vouchers & Web Check-In), FR-5 (Room Inventory & Maintenance Blackouts), FR-6 (ADR/RevPAR Financial Analytics & Audit Logs).
- **Non-Functional:** Strict ACID concurrency via `PESSIMISTIC_WRITE` row locking, BCrypt salted password hashing, sub-500ms catalog queries, and 100% open-source map integration with zero API billing risks.

2. Week 3 Deliverable: Production UML System Architecture

Below are the actual, verified UML diagrams modeled directly after the StayEase codebase and business logic:

Figure 1: StayEase Use Case Diagram (Actors & System Capabilities) — UML BEHAVIORAL

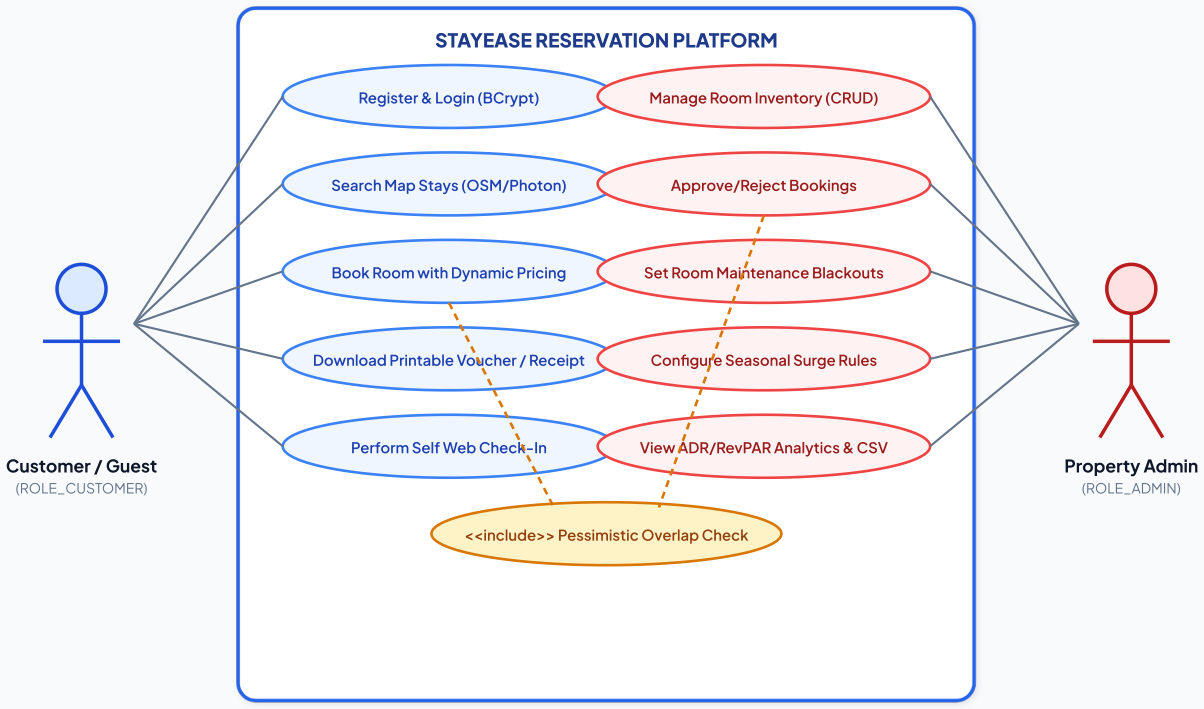


Figure 2: StayEase Structural Class Diagram (JPA Entities, Attributes & Associations) — UML STRUCTURAL

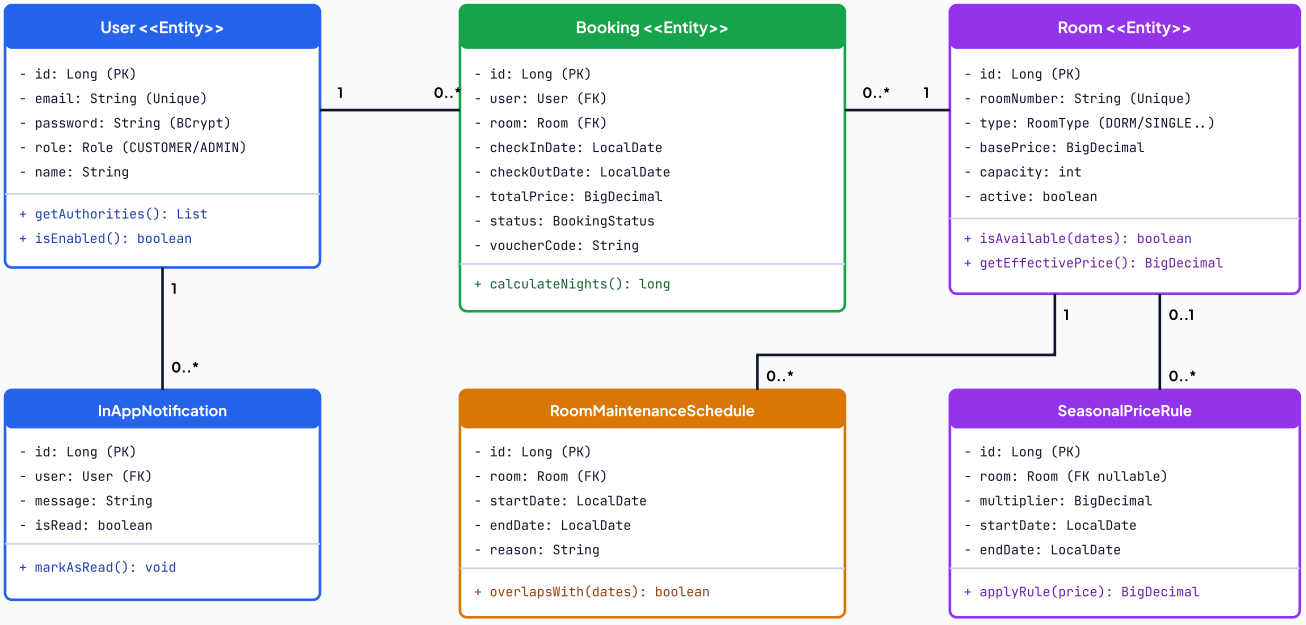


Figure 3: Sequence Diagram — Pessimistic Row Locking & Overlap Validation

UML DYNAMIC INTERACTION

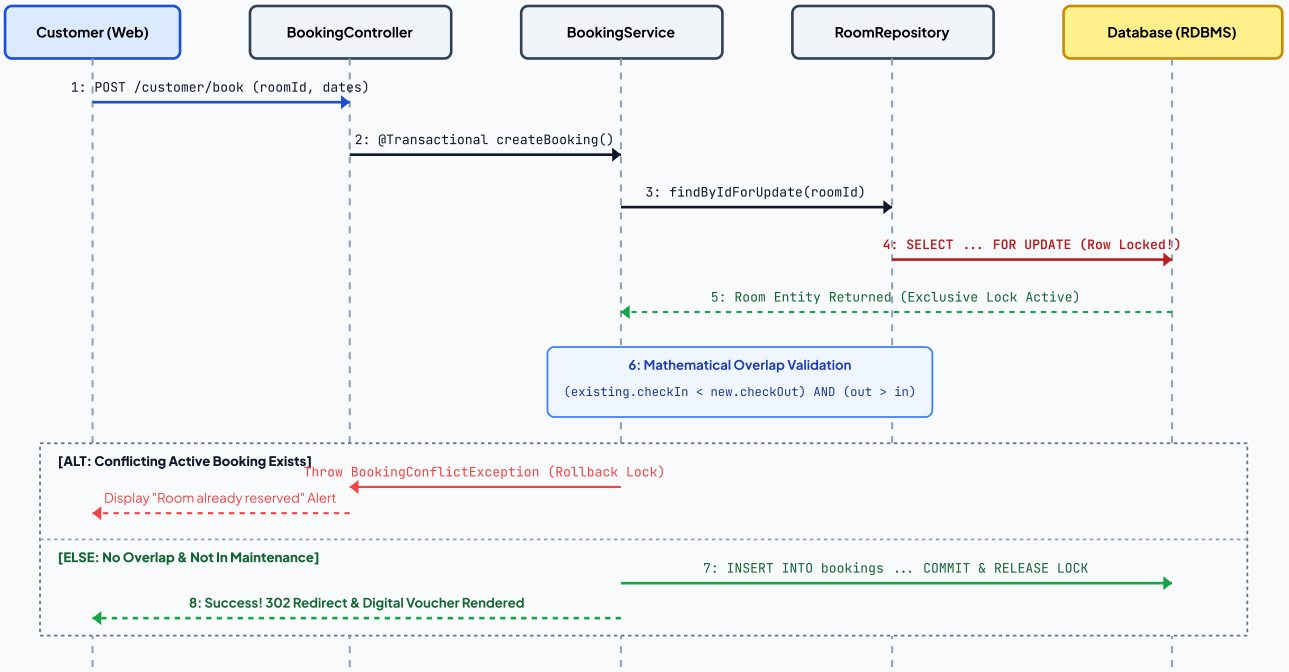
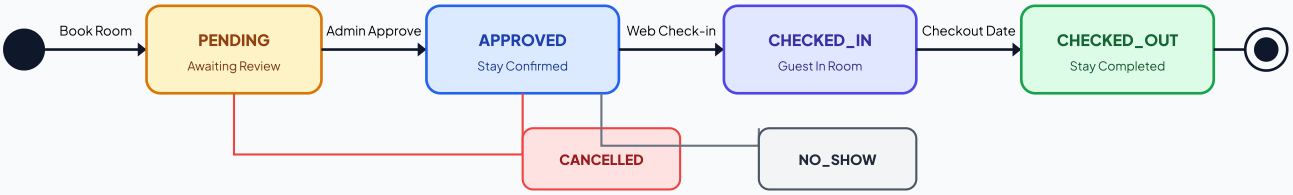


Figure 4: Booking Lifecycle State Machine Diagram

UML BEHAVIORAL STATE



3. Week 4 Deliverable: 3NF Database Design & Entity–Relationship Diagram

The relational schema was designed and normalized to Third Normal Form (3NF) to guarantee transactional ACID compliance and zero redundancy:

Figure 5: StayEase 3NF Entity–Relationship (ER) Schema Diagram — DATABASE SCHEMA

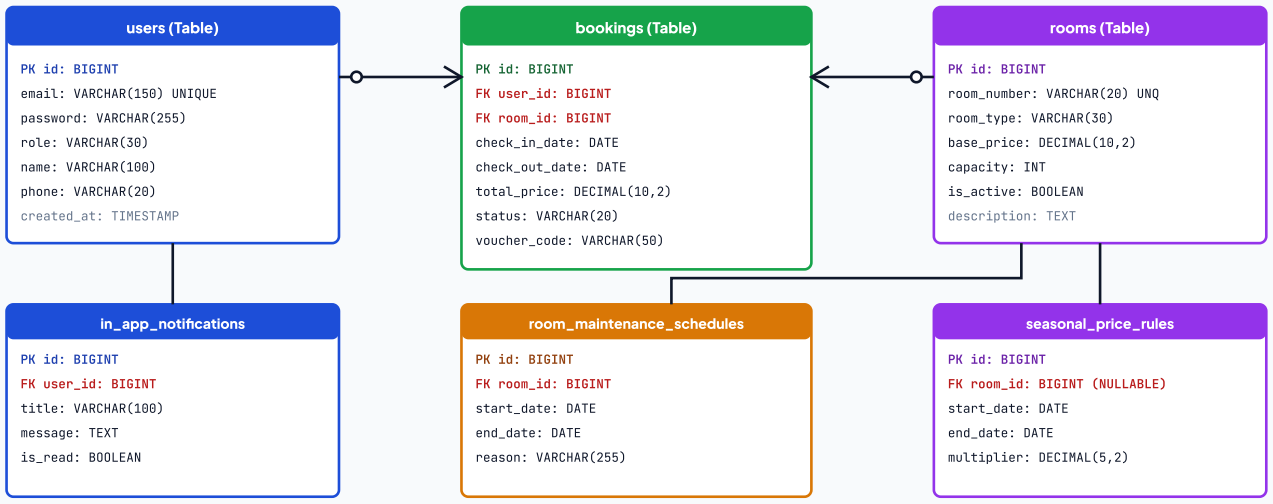
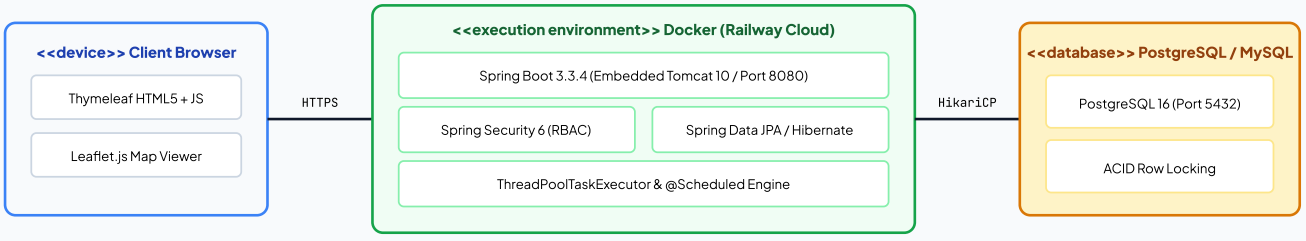


Figure 6: Component & Cloud Deployment Architecture — UML PHYSICAL DEPLOYMENT



4. Weeks 5–8: Module Ownership (4 Students = 4 Modules)

MODULE 1 (STUDENT 1): Worldwide Geospatial Discovery & Open Map Engine

Engineered Leaflet.js interactive pins, Komoot Photon 220ms debounced typeahead autocomplete, Haversine spherical formula in Java (`calculateDistanceKm`), and dual-tier Overpass/Nominatim bounding box scanning up to 100 km with zero API cost.

MODULE 2 (STUDENT 2): Customer Booking Engine & Concurrency Control

Implemented database `PESSIMISTIC_WRITE` row locking (`SELECT ... FOR UPDATE`) on rooms, canonical date overlap algorithm `(existing.checkIn < new.checkOut) AND (existing.checkOut > new.checkIn)` , same-day back-to-back stays, and digital printable voucher receipts.

MODULE 3 (STUDENT 3): Property Operations, Revenue Analytics & Compliance Audit

Built the Room Maintenance Blackout scheduler (repairs blocking room availability), Seasonal Price rules, **ADR (Average Daily Rate)** and **RevPAR (Revenue Per Available Room)** computations, and immutable compliance CSV audit trail export.

MODULE 4 (STUDENT 4): Authentication, Spring Security, Notifications & Cron Reminders

Engineered Spring Security 6 RBAC (`ROLE_CUSTOMER` vs `ROLE_ADMIN`), BCrypt salted password hashing, CSRF synchronizer tokens, `ThreadPoolTaskExecutor` async emails, and `@Scheduled` daily check-in reminder alerts.

5. Week 9: System Integration, 295+ Automated Tests & Cloud Deployment

Test Suite Layer	Framework Used	What Was Verified & Proved	Status
Unit Testing	JUnit 5, AssertJ	Haversine spherical distance calculation, pricing multipliers, form DTO validations.	100% Pass
Integration Testing	Spring Boot Test, H2	Spring Data JPA repositories, database relations, foreign key cascades.	100% Pass
Concurrency Testing	Multithreaded Executor	Flash crowd race conditions; verified `PESSIMISTIC_WRITE` lock prevents double-booking.	Zero Overbooking
Security Testing	MockMvc	Verified 403 Forbidden on illegal customer access to `/admin/**` , CSRF validation.	100% Pass

Docker & Cloud Deployment: Multi-stage `Dockerfile` packaging Eclipse Temurin Java 21 JRE, orchestrated via `docker-compose.yml` with MySQL 8, deployed live to Railway cloud with automated GitHub CI/CD synchronization.